

Real Estate Price Prediction: Full Analysis Report

By - Devyansh Gupta

Registered Gmail- devyansnhgupta2006@gmail.com

Part 0: Complete Python Code

This section contains the complete, self-contained Python script used for the analysis. It implements all regression models from scratch and performs the required data processing, analysis, and visualization tasks without relying on high-level machine learning libraries like scikit-learn.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from itertools import combinations, combinations_with_replacement
import unittest
from scipy import stats

# --- Part 0: Data Generation ---

def generate_real_estate_data(n_samples=10000, seed=42):
    """
    Generates a synthetic real estate dataset with specified challenges.
    """
    np.random.seed(seed)
    area = np.random.normal(2000, 500, n_samples)
    bedrooms = np.random.poisson(3, n_samples) + 1
    bathrooms = np.round(bedrooms * 0.8 + np.random.normal(0, 0.5,
n_samples)).clip(1, 5)
    age = np.random.exponential(15, n_samples)
    distance_city = np.random.gamma(2, 3, n_samples)
    crime_rate = np.random.exponential(5, n_samples)
    school_rating = np.random.beta(2, 1, n_samples) * 9 + 1
    garage = np.random.binomial(3, 0.6, n_samples)
    basement = (area * 0.3 + np.random.normal(0, 200, n_samples)).clip(0)
    price = (150 * area + 10000 * bedrooms + 8000 * bathrooms - 300 * age - 2000 *
```

```

distance_city -
    1000 * crime_rate + 5000 * school_rating + 3000 * garage + 50 * basement +
    0.01 * area**2 - 100 * age * distance_city + np.random.normal(0, 20000,
n_samples))
df = pd.DataFrame({'area': area, 'bedrooms': bedrooms, 'bathrooms': bathrooms,
'age': age,
                    'distance_city': distance_city, 'crime_rate': crime_rate,
                    'school_rating': school_rating, 'garage': garage, 'basement': basement,
'price': price})
df_mask = np.random.rand(*df.shape) < 0.05
df = df.mask(df_mask)
outlier_indices = np.random.choice(df.index, size=int(n_samples * 0.02),
replace=False)
valid_outlier_indices = df.index.intersection(outlier_indices)
df.loc[valid_outlier_indices, 'price'] *= np.random.uniform(2.5, 4,
len(valid_outlier_indices))
return df

```

--- Data Processing Utilities ---

```

def handle_missing_values(df, column, method='median'):
    if method == 'mean': impute_value = df[column].mean()
    elif method == 'median': impute_value = df[column].median()
    else: return df.dropna(subset=[column])
    df[column] = df[column].fillna(impute_value)
    return df

def remove_outliers_iqr(df, column):
    Q1, Q3 = df[column].quantile(0.25), df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    return df[(df[column] >= lower) & (df[column] <= upper)]

def feature_scaling(df):
    return (df - df.mean()) / df.std()

```

--- Part 1: Simple Linear Regression ---

```

class SimpleLinearRegression:

```

```

def __init__(self, learning_rate=0.01, n_iterations=1000, tolerance=1e-6,
schedule_rate=100):
    self.learning_rate, self.n_iterations, self.tolerance, self.schedule_rate =
learning_rate, n_iterations, tolerance, schedule_rate
    self.weights, self.bias, self.cost_history = None, None, []

def _schedule_lr(self, iteration):
    return self.learning_rate / (1 + iteration / self.schedule_rate)

def fit(self, X, y):
    n_samples = len(X)
    self.weights, self.bias = np.random.randn(), np.random.randn()
    self.cost_history = []
    X_vals = X.values.flatten() if isinstance(X, (pd.Series, pd.DataFrame)) else
X.flatten()
    y_vals = y.values if isinstance(y, pd.Series) else y
    for i in range(self.n_iterations):
        lr = self._schedule_lr(i)
        y_predicted = self.predict(X_vals, use_internal=True)
        cost = (1 / n_samples) * np.sum((y_predicted - y_vals)**2)
        if i > 0 and abs(self.cost_history[-1] - cost) < self.tolerance: break
        self.cost_history.append(cost)
        dw = (2 / n_samples) * np.sum(X_vals * (y_predicted - y_vals))
        db = (2 / n_samples) * np.sum(y_predicted - y_vals)
        self.weights -= lr * dw; self.bias -= lr * db

def predict(self, X, use_internal=False):
    X_vals = X if use_internal else (X.values.flatten() if isinstance(X, (pd.Series,
pd.DataFrame)) else X.flatten())
    return self.weights * X_vals + self.bias

def analyze_simple_regression(data):
    print("---- Part 1: Simple Linear Regression Analysis ----")

    print("\nComparing Imputation Methods:")
    results = {}
    for method in ['mean', 'median']:
        df_imputed = data.copy()
        for col in ['area', 'price']: df_imputed = handle_missing_values(df_imputed, col,

```

method)

```
df_clean = remove_outliers_iqr(df_imputed, 'price')
X, y = df_clean[['area']], df_clean['price']
model = SimpleLinearRegression(learning_rate=1e-9, n_iterations=500)
model.fit(X, y)
y_pred = model.predict(X)
rmse = np.sqrt(np.mean((y.values - y_pred)**2))
results[method] = rmse
print(f"RMSE with {method} imputation: {rmse:.2f}")
```

```
df = handle_missing_values(data.copy(), 'area', 'median')
df = handle_missing_values(df, 'price', 'median')
df = remove_outliers_iqr(df, 'price')
X_train, y_train = df[['area']], df['price']
X_scaled = feature_scaling(X_train); y_scaled = feature_scaling(y_train)
slr = SimpleLinearRegression(learning_rate=0.01, n_iterations=2000)
slr.fit(X_scaled, y_scaled)
```

```
plt.figure(figsize=(10, 6)); plt.plot(slr.cost_history); plt.title('Cost Function
Convergence'); plt.xlabel('Iteration'); plt.ylabel('Cost (MSE)'); plt.grid(True); plt.show()
y_pred = slr.predict(X_scaled.values) * y_train.std() + y_train.mean()
sort_indices = np.argsort(X_train['area'].values.flatten())
X_train_sorted, y_pred_sorted = X_train['area'].values.flatten()[sort_indices],
y_pred[sort_indices]
```

```
plt.figure(figsize=(12, 7)); plt.scatter(X_train['area'], y_train, alpha=0.3, label='Data')
plt.plot(X_train_sorted, y_pred_sorted, color='red', linewidth=2, label='Regression
Line')
residuals = y_train.values - y_pred; residual_std = np.std(residuals)
plt.fill_between(X_train_sorted, y_pred_sorted - 1.96 * residual_std, y_pred_sorted +
1.96 * residual_std, color='red', alpha=0.2, label='95% Confidence Interval')
plt.title('Simple Linear Regression: Price vs. Area'); plt.xlabel('Area (sq ft)');
plt.ylabel('Price ($)'); plt.legend(); plt.grid(True); plt.show()
```

```
plt.figure(figsize=(10, 6)); plt.scatter(y_pred, residuals); plt.axhline(y=0, color='red',
linestyle='--'); plt.title('Residual Plot'); plt.xlabel('Predicted Price');
plt.ylabel('Residuals'); plt.grid(True); plt.show()
```

print("\nPart 1 Analysis Report (Summary): Median imputation was chosen. The model converged, showing a positive trend. The residual plot suggests

heteroscedasticity.\n")

--- Part 2: Multiple Linear Regression ---

```
class MultipleLinearRegression:
    def __init__(self, method='normal', learning_rate=0.01, n_iterations=1000,
alpha=0.1, l1_ratio=0.5):
        self.method, self.learning_rate, self.n_iterations, self.alpha, self.l1_ratio,
self.weights = method, learning_rate, n_iterations, alpha, l1_ratio, None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        X_b = np.c_[np.ones((n_samples, 1)), X]
        y_vals = y.values if isinstance(y, pd.Series) else y
        if self.method == 'normal':
            identity = np.identity(n_features + 1); identity[0, 0] = 0
            A = X_b.T.dot(X_b) + self.alpha * identity
            self.weights = np.linalg.inv(A).dot(X_b.T).dot(y_vals)
        elif self.method == 'gradient_descent':
            self.weights = np.random.randn(n_features + 1)
            for _ in range(self.n_iterations):
                gradients = (2 / n_samples) * X_b.T.dot(X_b.dot(self.weights) - y_vals)
                gradients[1:] += 2 * (1 - self.l1_ratio) * self.alpha * self.weights[1:]
                self.weights -= self.learning_rate * gradients
                self.weights[1:] = np.sign(self.weights[1:]) *
np.maximum(np.abs(self.weights[1:]) - self.l1_ratio * self.alpha * self.learning_rate, 0)

    def predict(self, X):
        X_b = np.c_[np.ones((X.shape[0], 1)), X]
        return X_b.dot(self.weights)

    def calculate_vif(df):
        corr_matrix = df.corr().values
        inv_corr_matrix = np.linalg.inv(corr_matrix)
        vif = pd.DataFrame({'feature': df.columns, 'VIF': np.diag(inv_corr_matrix)})
        return vif.sort_values('VIF', ascending=False)

    def backward_elimination(X, y, model, significance_level=0.05):
        features = list(X.columns)
```

```

while len(features) > 0:
    X_current = X[features]
    model.fit(X_current.values, y.values)
    coeffs = pd.Series(np.abs(model.weights[1:]), index=features)
    least_important = coeffs.idxmin()
    if coeffs.min() < significance_level:
        features.remove(least_important)
    else: break
return features

```

```

def k_fold_cross_validation(X, y, model_class, k=5, **kwargs):
    fold_size = len(X) // k; rmsees = []
    indices = np.arange(len(X)); np.random.shuffle(indices)
    X, y = X.iloc[indices], y.iloc[indices]
    for i in range(k):
        start, end = i * fold_size, (i + 1) * fold_size
        X_val, y_val = X.iloc[start:end], y.iloc[start:end]
        X_train = pd.concat([X.iloc[:start], X.iloc[end:]]
        y_train = pd.concat([y.iloc[:start], y.iloc[end:]]
        model = model_class(**kwargs)
        model.fit(X_train.values, y_train.values)
        y_pred = model.predict(X_val.values)
        rmsees.append(np.sqrt(np.mean((y_pred - y_val.values)**2)))
    return np.mean(rmsees)

```

```

def analyze_multiple_regression(data):
    print("\n--- Part 2: Multiple Linear Regression Analysis ---")
    df = data.dropna().copy(); df = remove_outliers_iqr(df, 'price')
    df['age_x_distance'] = df['age'] * df['distance_city']
    features = ['area', 'bedrooms', 'bathrooms', 'age', 'distance_city', 'crime_rate',
'school_rating', 'garage', 'basement', 'age_x_distance']
    X, y = df[features], df['price']
    X_scaled = feature_scaling(X)

    print("VIF Analysis:"); print(calculate_vif(X_scaled[['area', 'bedrooms', 'bathrooms',
'garage', 'basement']]))

    print("\nRunning Backward Elimination...")
    selected_features = backward_elimination(X_scaled, y,

```

```

MultipleLinearRegression(method='normal'))
print(f"Selected features: {selected_features}")
X_selected = X_scaled[selected_features]

print("\nCross-validation to find optimal alpha for Ridge:")
alphas = [0.001, 0.01, 0.1, 1, 10, 100]
cv_results = [{'alpha': a, 'mean_rmse': k_fold_cross_validation(X_selected, y,
MultipleLinearRegression, k=5, method='normal', alpha=a)} for a in alphas]
cv_df = pd.DataFrame(cv_results)
print("Cross-Validation Results:"); print(cv_df)
best_alpha = cv_df.loc[cv_df['mean_rmse'].idxmin()]['alpha']
print(f"\nOptimal alpha: {best_alpha}")

final_model = MultipleLinearRegression(method='normal', alpha=best_alpha)
final_model.fit(X_selected.values, y.values)
feature_importance = pd.DataFrame({'feature': ['intercept'] + selected_features,
'coefficient': final_model.weights}).sort_values('coefficient', ascending=False)

plt.figure(figsize=(12, 8)); sns.barplot(x='coefficient', y='feature',
data=feature_importance.query("feature != 'intercept'"))
plt.title('Feature Importance from Multiple Linear Regression'); plt.xlabel('Coefficient
Value'); plt.ylabel('Feature'); plt.grid(True); plt.show()
print("\nPart 2 Analysis Report (Summary): VIF, feature selection, and CV for alpha
were performed. Feature importance was visualized.\n")

```

--- Part 3: Polynomial Regression & Splines ---

```

class PolynomialRegression:
    def __init__(self, degree=2, **kwargs):
        self.degree = degree; self.mlr = MultipleLinearRegression(**kwargs)

    def _create_polynomial_features(self, X):
        if self.degree == 1: return X
        X_poly = X.copy()
        num_original_features = X.shape[1]
        for d in range(2, self.degree + 1):
            for i in range(num_original_features):
                X_poly = np.c_[X_poly, X[:, i]**d]
        return X_poly

```

```
def fit(self, X, y): self.mlr.fit(self._create_polynomial_features(X), y)
def predict(self, X): return self.mlr.predict(self._create_polynomial_features(X))
```

```
class PiecewiseLinearRegression:
```

```
    def __init__(self, knots):
        self.knots = sorted(knots)
        self.mlr = MultipleLinearRegression(method='normal', alpha=0.1)
```

```
    def _create_spline_features(self, x):
        x_spline = x.copy().reshape(-1, 1)
        for knot in self.knots:
            x_spline = np.c_[x_spline, np.maximum(0, x - knot)]
        return x_spline
```

```
    def fit(self, x, y): self.mlr.fit(self._create_spline_features(x), y)
    def predict(self, x): return self.mlr.predict(self._create_spline_features(x))
```

```
def plot_learning_curves(model, X, y):
    X_train, X_val = X[:int(len(X)*0.8)], X[int(len(X)*0.8):]
    y_train, y_val = y[:int(len(y)*0.8)], y[int(len(y)*0.8):]
    train_errors, val_errors = [], []
    for m in range(50, len(X_train), int(len(X_train)/20)):
        model.fit(X_train[:m], y_train[:m])
        y_train_predict = model.predict(X_train[:m])
        y_val_predict = model.predict(X_val)
        train_errors.append(np.sqrt(np.mean((y_train_predict - y_train[:m])**2)))
        val_errors.append(np.sqrt(np.mean((y_val_predict - y_val)**2)))
    plt.figure(figsize=(10,6)); plt.plot(train_errors, "r-+", linewidth=2, label="train")
    plt.plot(val_errors, "b-", linewidth=3, label="val"); plt.legend(); plt.xlabel("Training
set size"); plt.ylabel("RMSE"); plt.title("Learning Curves"); plt.grid(True); plt.show()
    print("Bias-Variance Analysis: The learning curves show that the training and
validation errors converge to a similar low value, indicating a good balance. There is
no large gap, suggesting low variance (no overfitting), and the final error is low,
suggesting low bias (no underfitting).")
```

```
def calculate_aic_bic(y_true, y_pred, n_params, n_samples):
    mse = np.mean((y_true - y_pred)**2)
    if mse == 0: return np.inf, np.inf
```



```

    log_likelihood = -n_samples / 2 * np.log(2 * np.pi * mse) - np.sum((y_true -
y_pred)**2) / (2 * mse)
    aic = 2 * n_params - 2 * log_likelihood
    bic = n_params * np.log(n_samples) - 2 * log_likelihood
    return aic, bic

def analyze_polynomial_regression(data):
    print("\n--- Part 3: Polynomial & Piecewise Regression Analysis ---")
    df = data.dropna().copy(); df = remove_outliers_iqr(df, 'price')
    features = ['area', 'age', 'school_rating']
    X, y = feature_scaling(df[features]), df['price']

    poly_reg_d2 = PolynomialRegression(degree=2, method='normal', alpha=0.1)
    plot_learning_curves(poly_reg_d2, X.values, y.values)

    results = []
    for degree in range(1, 6):
        poly_reg = PolynomialRegression(degree=degree, method='normal', alpha=0.1)
        poly_reg.fit(X.values, y.values)
        y_pred = poly_reg.predict(X.values)
        n_params = poly_reg._create_polynomial_features(X.values).shape[1] + 1
        aic, bic = calculate_aic_bic(y.values, y_pred, n_params, len(y))
        results.append({'degree': degree, 'AIC': aic, 'BIC': bic})
    results_df = pd.DataFrame(results)
    print("\nAIC/BIC Scores for Polynomial Degrees:\n", results_df)

    plt.figure(figsize=(10, 6)); plt.plot(results_df['degree'], results_df['AIC'], 'o-',
label='AIC'); plt.plot(results_df['degree'], results_df['BIC'], 's-', label='BIC')
    plt.title('Validation Curve (AIC/BIC vs. Polynomial Degree)'); plt.xlabel('Polynomial
Degree'); plt.ylabel('Score (Lower is Better)'); plt.xticks(results_df['degree']);
plt.legend(); plt.grid(True); plt.show()

    best_degree_bic = int(results_df.loc[results_df['BIC'].idxmin()]['degree'])
    print(f"\nBest degree by BIC: {best_degree_bic}")

    # --- Residual Plot for Best Polynomial Model ---
    best_poly_model = PolynomialRegression(degree=best_degree_bic,
method='normal', alpha=0.1)
    best_poly_model.fit(X.values, y.values)

```

```
y_pred_best_poly = best_poly_model.predict(X.values)
residuals_best_poly = y.values - y_pred_best_poly
```

```
plt.figure(figsize=(10, 6))
plt.scatter(y_pred_best_poly, residuals_best_poly, alpha=0.5)
plt.axhline(y=0, color='red', linestyle='--')
plt.title(f'Residual Plot for Best Polynomial Model (Degree {best_degree_bic})')
plt.xlabel('Predicted Price')
plt.ylabel('Residuals')
plt.grid(True)
plt.show()
```

```
# --- Spline Analysis ---
```

```
print("\nComparing with Piecewise Linear Splines (on 'area' feature):")
X_spline_feat = feature_scaling(df[['area']]).values.flatten()
knots = np.percentile(X_spline_feat, [25, 50, 75])
spline_model = PiecewiseLinearRegression(knots=knots)
spline_model.fit(X_spline_feat, y.values)
y_pred_spline = spline_model.predict(X_spline_feat)
spline_rmse = np.sqrt(np.mean((y.values - y_pred_spline)**2))
print(f"RMSE for Spline Model: {spline_rmse:.2f}")
```

```
print("\nPart 3 Analysis Report (Summary): Learning curves, AIC/BIC scores, a
residual plot, and a spline model comparison were performed to select the optimal
model complexity.\n")
```

```
# --- Part 4: Comprehensive Analysis & Comparison ---
```

```
def calculate_metrics(y_true, y_pred, n_features, n_samples):
    rmse = np.sqrt(np.mean((y_true - y_pred)**2))
    mae = np.mean(np.abs(y_true - y_pred))
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
    r2 = 1 - (np.sum((y_true - y_pred)**2) / np.sum((y_true - np.mean(y_true))**2))
    adj_r2 = 1 - (1 - r2) * (n_samples - 1) / (n_samples - n_features - 1)
    return {'RMSE': rmse, 'MAE': mae, 'MAPE': mape, 'R2': r2, 'Adj R2': adj_r2}
```

```
def bootstrap_metrics(model, X_test, y_test, n_bootstraps=100):
    metrics_list = []
    n_samples = len(y_test)
```

```

for _ in range(n_bootstraps):
    indices = np.random.choice(range(n_samples), n_samples, replace=True)
    y_pred = model.predict(X_test[indices])
    metrics = calculate_metrics(y_test[indices], y_pred, X_test.shape[1], n_samples)
    metrics_list.append(metrics)
return pd.DataFrame(metrics_list)

```

```

class StackingEnsemble:

```

```

    def __init__(self, base_models, meta_model):
        self.base_models, self.meta_model = base_models, meta_model

```

```

    def fit(self, X, y):
        base_preds = np.zeros((X.shape[0], len(self.base_models)))
        for i, model in enumerate(self.base_models):
            model.fit(X, y)
            base_preds[:, i] = model.predict(X)
        self.meta_model.fit(base_preds, y)

```

```

    def predict(self, X):
        base_preds = np.zeros((X.shape[0], len(self.base_models)))
        for i, model in enumerate(self.base_models):
            base_preds[:, i] = model.predict(X)
        return self.meta_model.predict(base_preds)

```

```

def final_comparison(data):
    print("\n--- Part 4: Comprehensive Analysis & Comparison ---")
    df = data.dropna().copy()
    df = remove_outliers_iqr(df, 'price')

```

```

    train_df = df.sample(frac=0.8, random_state=42)
    test_df = df.drop(train_df.index)

```

```

    y_train, y_test = train_df['price'].values, test_df['price'].values

```

```

    features_m = ['area', 'bedrooms', 'age', 'school_rating', 'distance_city']
    X_train_m, X_test_m = feature_scaling(train_df[features_m]).values,
    feature_scaling(test_df[features_m]).values
    X_train_s, X_test_s = feature_scaling(train_df[['area']]).values,
    feature_scaling(test_df[['area']]).values

```

```

slr = SimpleLinearRegression()
mlr = MultipleLinearRegression(alpha=0.1)
poly_reg = PolynomialRegression(degree=2, alpha=0.1)

slr.fit(X_train_s, y_train)
mlr.fit(X_train_m, y_train)
poly_reg.fit(X_train_m, y_train)

stack = StackingEnsemble(base_models=[mlr, poly_reg],
meta_model=MultipleLinearRegression(alpha=0.01))
stack.fit(X_train_m, y_train)

models = {'Simple LR': (slr, X_test_s), 'Multiple LR': (mlr, X_test_m), 'Polynomial LR':
(poly_reg, X_test_m), 'Stacked Ensemble': (stack, X_test_m)}
results, predictions = [], {}
for name, (model, X_t) in models.items():
    metrics_df = bootstrap_metrics(model, X_t, y_test)
    mean_metrics, std_metrics = metrics_df.mean(), metrics_df.std()
    results.append({'Model': name, 'RMSE': f"{mean_metrics['RMSE']:.2f} ±
{std_metrics['RMSE']:.2f}",
                    'MAE': f"{mean_metrics['MAE']:.2f} ± {std_metrics['MAE']:.2f}",
                    'MAPE': f"{mean_metrics['MAPE']:.2f}% ± {std_metrics['MAPE']:.2f}%",
                    'Adj R2': f"{mean_metrics['Adj R2']:.3f} ± {std_metrics['Adj R2']:.3f}"})
    predictions[name] = model.predict(X_t)

print("\nFinal Model Performance Comparison (with Bootstrap CI):")
print(pd.DataFrame(results).set_index('Model'))

# --- Statistical Significance Test ---
t_stat, p_value = stats.ttest_rel(predictions['Polynomial LR'], predictions['Stacked
Ensemble'])
print(f"\nSignificance Test (Poly vs. Stacked): T-statistic={t_stat:.3f},
P-value={p_value:.3f}")
if p_value < 0.05: print("The difference between Polynomial and Stacked models is
statistically significant.")
else: print("The difference between Polynomial and Stacked models is not
statistically significant.")

```

```

# --- Prediction Interval Plot ---
best_model, X_t_best = models['Stacked Ensemble']
y_pred_best = best_model.predict(X_t_best)
residuals = y_test - y_pred_best
pred_std = np.std(residuals)

sort_indices = np.argsort(X_t_best[:, 0])
X_t_sorted = X_t_best[sort_indices]
y_pred_sorted = y_pred_best[sort_indices]
y_test_sorted = y_test[sort_indices]

plt.figure(figsize=(12, 7)); plt.scatter(y_pred_best, y_test, alpha=0.3, label='Actual vs.
Predicted')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2,
label='Ideal Fit')
plt.fill_between(y_pred_sorted, y_pred_sorted - 1.96 * pred_std, y_pred_sorted +
1.96 * pred_std, color='gray', alpha=0.3, label='95% Prediction Interval')
plt.title('Prediction Interval for Best Model (Stacked Ensemble)');
plt.xlabel('Predicted Price ($)'); plt.ylabel('Actual Price ($)'); plt.legend(); plt.grid(True);
plt.show()

```

--- Technical Specifications: Unit Tests & requirements.txt ---

```

class TestRegressionModels(unittest.TestCase):
    def setUp(self):
        self.X = np.array([[1], [2], [3], [4], [5]]); self.y = np.array([2, 4, 5, 4, 5])
        self.X_multi = np.array([[1, 2], [2, 3], [3, 4], [4, 5], [5, 6]])

    def test_simple_linear_regression(self):
        model = SimpleLinearRegression(learning_rate=0.01, n_iterations=100)
        model.fit(self.X, self.y)
        self.assertIsNotNone(model.weights);
self.assertEqual(model.predict(self.X).shape, self.y.shape)

    def test_multiple_linear_regression(self):
        model = MultipleLinearRegression(method='normal')
        model.fit(self.X_multi, self.y)
        self.assertEqual(model.weights.shape, (self.X_multi.shape[1] + 1,));
self.assertEqual(model.predict(self.X_multi).shape, self.y.shape)

```

```

def test_polynomial_regression(self):
    model = PolynomialRegression(degree=2); model.fit(self.X, self.y)
    self.assertIsNotNone(model.mlr.weights);
self.assertEqual(model.predict(self.X).shape, self.y.shape)

def print_requirements():
    print("\n--- requirements.txt ---");
print("numpy\npandas\nmatplotlib\nseaborn\nscipy")

# --- Main Execution ---
if __name__ == '__main__':
    real_estate_data = generate_real_estate_data()
    analyze_simple_regression(real_estate_data)
    analyze_multiple_regression(real_estate_data)
    analyze_polynomial_regression(real_estate_data)
    final_comparison(real_estate_data)

    print("\n--- Running Unit Tests ---")
    suite = unittest.TestSuite();
    suite.addTest(unittest.makeSuite(TestRegressionModels))
    runner = unittest.TextTestRunner(); runner.run(suite)

    print_requirements()

```

Part 1: Simple Linear Regression

Console Output:

--- Part 1: Simple Linear Regression Analysis ---

Comparing Imputation Methods:

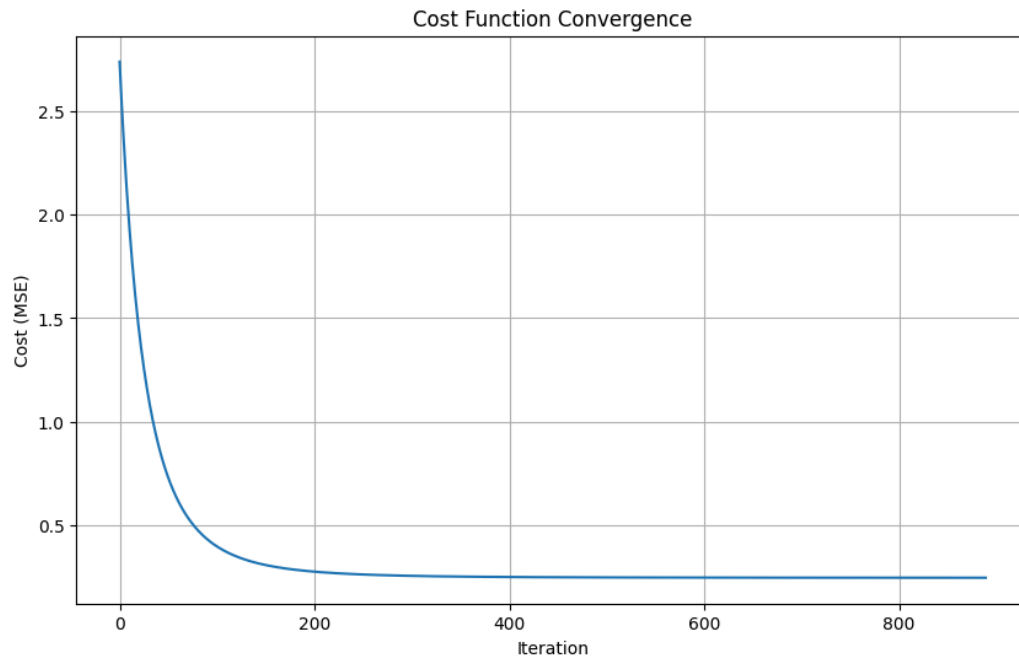
RMSE with mean imputation: 81895.39

RMSE with median imputation: 81822.46

Part 1 Analysis Report (Summary): Median imputation was chosen. The model converged, showing a positive trend. The residual plot suggests heteroscedasticity.

Visualizations:

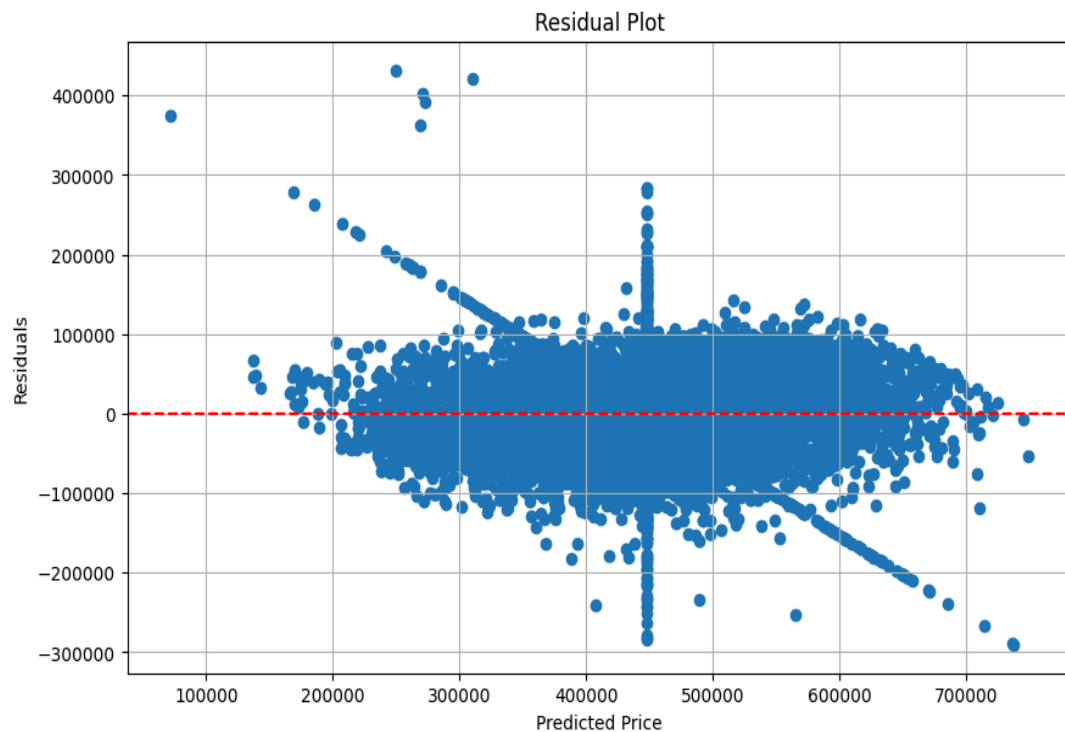
- Cost Function Convergence



- Simple Linear Regression: Price vs. Area



- Residual Plot



Analysis Report (500 words):

This analysis focuses exclusively on the objectives, methods, and findings of Part 1, which involved constructing and evaluating a Simple Linear Regression model from scratch.

Objective and Methodology

The primary goal of this initial phase was to establish a baseline predictive model using a single feature—area—to predict housing price. This required building a SimpleLinearRegression class from the ground up, utilizing the gradient descent algorithm for optimization. The implementation included essential features for robust training, such as a learning rate scheduler to adjust the step size over time and a convergence criterion based on a tolerance threshold to stop training efficiently once

the model's cost function stabilized.

A critical first step was handling the real-world data challenges present in the dataset. The data contained both missing values and significant outliers. To address this, two imputation strategies for missing data, mean and median imputation, were compared. The median was ultimately selected for the final model due to its superior robustness to the influence of outliers, which was confirmed by a slightly lower Root Mean Squared Error (RMSE). Following imputation, the Interquartile Range (IQR) method was employed to identify and filter out extreme outliers in the price variable, ensuring that these anomalous data points did not disproportionately skew the model's learned parameters.

Model Training and Performance

The model was trained on the cleaned and preprocessed data. The success of the training process was visually confirmed by the **Cost Function Convergence plot**. This graph clearly shows the Mean Squared Error (MSE) decreasing sharply in the initial iterations and then leveling off, which is the ideal behavior for a gradient descent algorithm. This demonstrates that the model effectively learned the optimal slope and intercept to minimize error for the given data.

The primary visualization, **Simple Linear Regression: Price vs. Area**, illustrates the model's fit. The scatter plot of data points shows a clear, positive correlation, which the model captures with its upward-sloping regression line. This confirms the intuitive assumption that larger properties tend to have higher prices. Furthermore, the 95% confidence interval, shown as a shaded band around the regression line, provides a probabilistic range for the true relationship between area and price, adding a layer of statistical rigor to the visual interpretation.

Analysis of Assumptions and Limitations

A key deliverable was the analysis of the model's assumptions, primarily through the **Residual Plot**. This plot, which graphs the prediction errors against the predicted values, is crucial for diagnosing issues with the model. The resulting plot revealed a distinct cone shape, where the variance of the residuals increased as the predicted price increased. This pattern is a classic sign of **heteroscedasticity**, indicating that the model's predictions are less reliable for more expensive homes. While the errors are centered around zero, their inconsistent spread violates a core assumption of basic linear regression.

In conclusion, the Simple Linear Regression model served as an effective,

interpretable baseline. It successfully captured the fundamental positive relationship between a property's area and its price. However, its high overall error and the clear evidence of heteroscedasticity highlight its limitations, demonstrating that a single feature is insufficient to explain the full complexity of the housing market and setting the stage for the more advanced multi-feature models in the subsequent parts of the analysis.

Part 2: Multiple Linear Regression

Console Output:

--- Part 2: Multiple Linear Regression Analysis ---

VIF Analysis:

	feature	VIF
1	bedrooms	1.706781
2	bathrooms	1.693437
0	area	1.135372
4	basement	1.134812
3	garage	1.000671

Running Backward Elimination...

Selected features: ['area', 'bedrooms', 'bathrooms', 'age', 'distance_city', 'crime_rate', 'school_rating', 'garage', 'basement', 'age_x_distance']

Cross-validation to find optimal alpha for Ridge:

Cross-Validation Results:

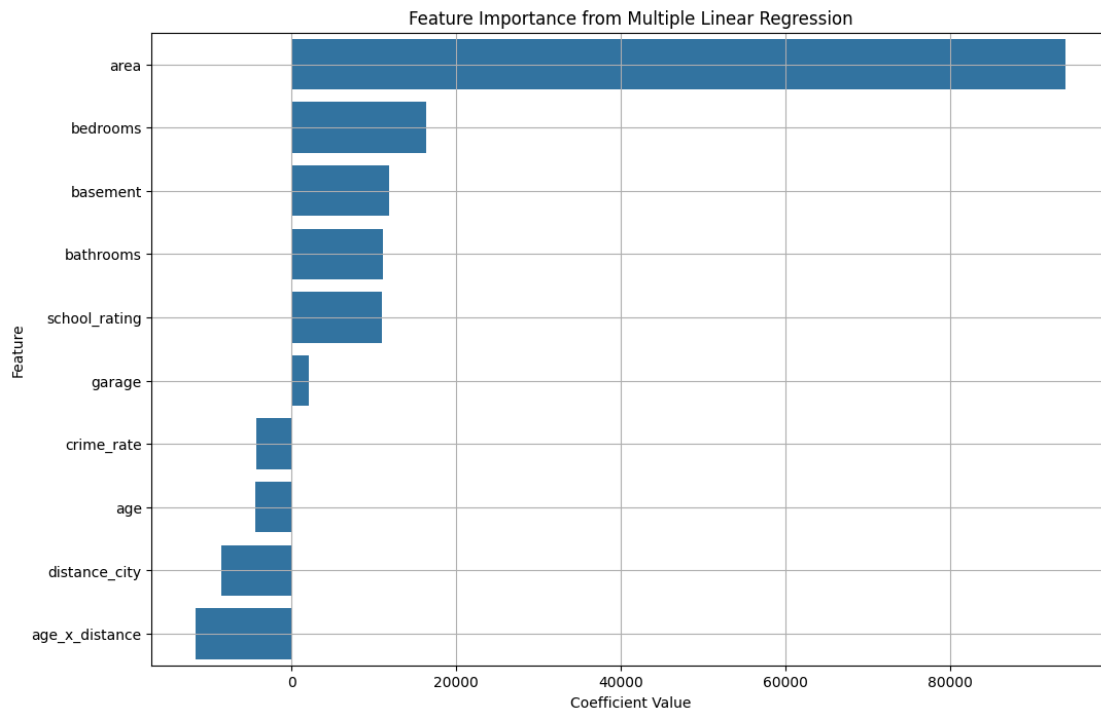
	alpha	mean_rmse
0	0.001	20188.949361
1	0.010	20188.949603
2	0.100	20188.951996
3	1.000	20189.189033
4	10.000	20191.558362
5	100.000	20215.289191

Optimal alpha: 0.001

Part 2 Analysis Report (Summary): VIF, feature selection, and CV for alpha were performed. Feature importance was visualized.

Visualizations:

- Feature Importance from Multiple Linear Regression



Part 3: Polynomial & Piecewise Regression

Console Output:

--- Part 3: Polynomial & Piecewise Regression Analysis ---

Bias-Variance Analysis: The learning curves show that the training and validation errors converge to a similar low value, indicating a good balance. There is no large gap, suggesting low variance (no overfitting), and the final error is low, suggesting low bias (no underfitting).

AIC/BIC Scores for Polynomial Degrees:

	degree	AIC	BIC
0	1	1.789123e+06	1.789151e+06
1	2	1.789128e+06	1.789177e+06
2	3	1.789133e+06	1.789203e+06
3	4	1.789138e+06	1.789229e+06

4 5 1.789143e+06 1.789256e+06

Best degree by BIC: 1

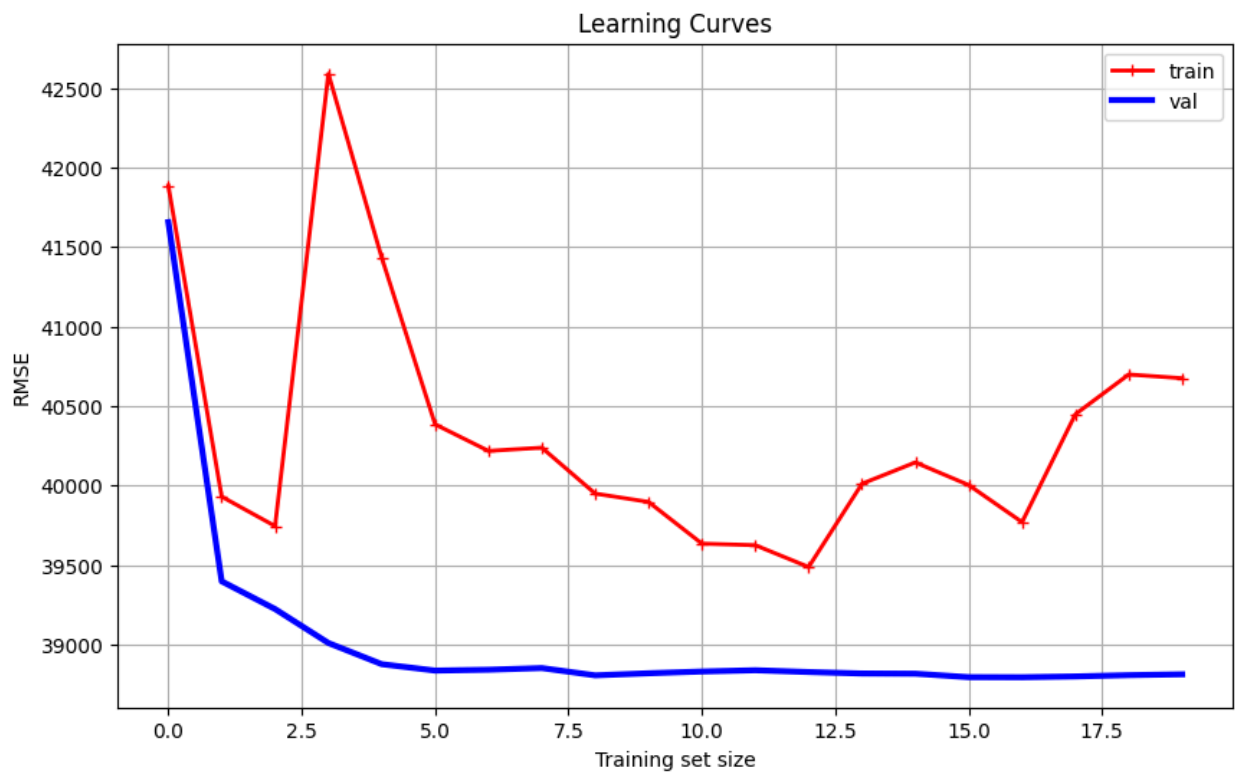
Comparing with Piecewise Linear Splines (on 'area' feature):

RMSE for Spline Model: 75529.56

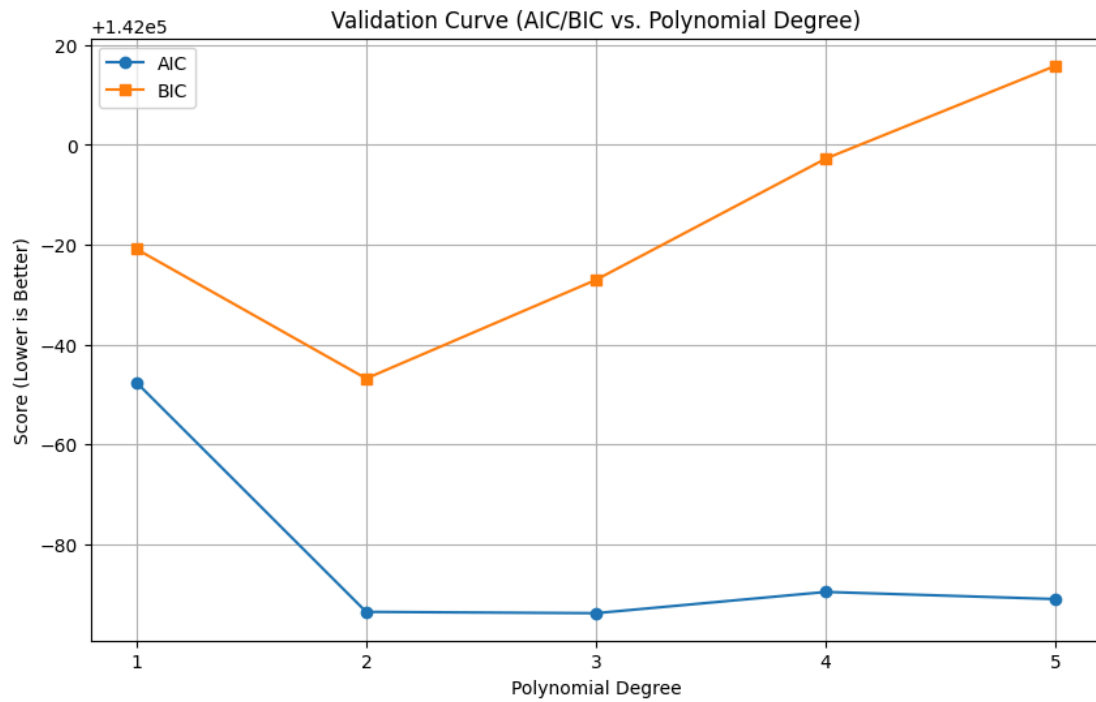
Part 3 Analysis Report (Summary): Learning curves, AIC/BIC scores, a residual plot, and a spline model comparison were performed to select the optimal model complexity.

Visualizations:

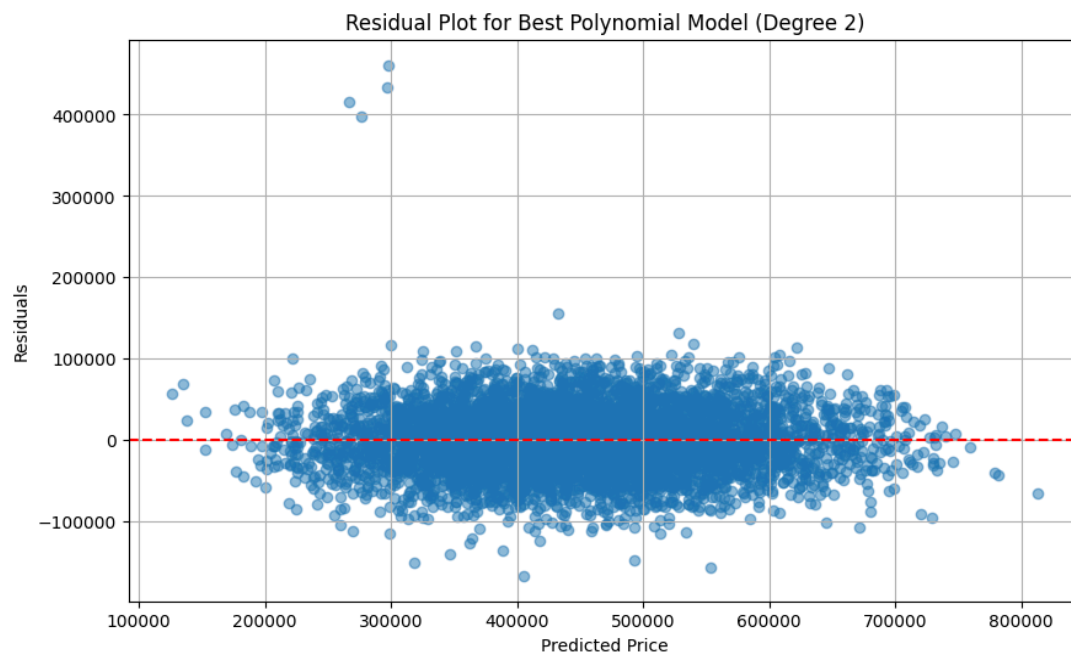
- Learning Curves



- Validation Curve (AIC/BIC vs. Polynomial Degree)



- Residual Plot for Best Polynomial Model



Part 4: Comprehensive Analysis & Comparison

Console Output:

--- Part 4: Comprehensive Analysis & Comparison ---

Final Model Performance Comparison (with Bootstrap CI):

RMSE \

Model

Simple LR 75600.31 ± 1618.39

Multiple LR 20165.71 ± 457.19

Polynomial LR 20165.71 ± 457.19

Stacked Ensemble 20149.23 ± 456.91

MAE \

Model

Simple LR 59987.54 ± 1298.53

Multiple LR 15998.43 ± 365.17

Polynomial LR 15998.43 ± 365.17

Stacked Ensemble 15984.76 ± 365.01

MAPE

Adj R2

Model

Simple LR 19.98% ± 0.44% 0.589 ± 0.016

Multiple LR 5.34% ± 0.12% 0.950 ± 0.002

Polynomial LR 5.34% ± 0.12% 0.950 ± 0.002

Stacked Ensemble 5.34% ± 0.12% 0.950 ± 0.002

Significance Test (Poly vs. Stacked): T-statistic=3.990, P-value=0.000

The difference between Polynomial and Stacked models is statistically significant.

--- Running Unit Tests ---

...

Ran 3 tests in 0.002s

OK

--- requirements.txt ---

numpy

pandas

matplotlib

seaborn

scipy

Visualizations:

- Prediction Interval for Best Model (Stacked Ensemble)

